

Object-Relational Mapping

Parte 2 - Mapeamento de Relações

Mapeamento de Atributos

Agregação de dados 1-1

@Entity

```
public class Author {
```

@Embedded

```
private Address address;
```

@Embeddable

```
public class Address {
```

```
private String street;
```

```
private String city;
```

```
private String zipCode;
```

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.

Padrão **Embeddable Value**:

- Quando classes não são entidades por si mesmas, podemos colocar como campos compostos na entidade a que pertence (**@Embeddable**)

author Enter a SQL expression to filter results (use Ctrl+Space)							
Grid		123 id	A-Z city	A-Z street	A-Z zip_code	A-Z name	A-Z surname
1		1	Lisboa	Rua Exemplo	12345678	Jack	Bauer
2		2	Porto	Largo do ie	54243312	Chloe	O'Brian
3		3	Braga	Avenida Teste	74243312	Kim	Bauer
4		4	Setubal	Boulevard Aqui	92132132	David	Palmer
5		5	Lisboa	Quinta do Desisto	31245123	Michelle	Dessler

Mapeamento de Atributos

Agregação de dados 1-N

@Entity

```
public class Author {
```

@ElementCollection

```
private List<Address> addresses;
```

@Embeddable

```
public class Address {
```

```
private String street;
```

```
private String city;
```

```
private String zipCode;
```

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.

Padrão **Embeddable Value**:

- Quando classes não são entidades por si mesmas, podemos colocar como campos compostos na entidade a que pertence (**@Embeddable**)

author Enter a SQL expression to filter results				
Grid	id	name	surname	
1	1	Jack	Bauer	
2	2	Chloe	O'Brian	

author_addresses Enter a SQL expression to filter results (use Ctrl+Sp				
Grid	author_id	city	street	zip_code
1	1	Lisboa	Rua Exemplo	12345678
2	1	Cascais	Travessa Feliz	421312312
3	2	Porto	Largo do ie	54243312

Mapeamento de Atributos

Agregação de dados 1-N

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.

```
Author jack = new Author();
jack.setName("Jack");
jack.setSurname("Bauer");
Address mainHouse =
    new Address ( street: "Rua Exemplo", city: "Lisboa", zipCode: "12345678");
Address vacHouse =
    new Address( street: "Travessa Feliz", city: "Cascais", zipCode: "421312312");
jack.addAddress(mainHouse);
jack.addAddress(vacHouse);
authorRepository.save(jack);
```

Esta linha regista o objecto *jack* para mapeamento.

1. Cria a linha na tabela author

author Enter a SQL expression to filter results			
Grid	123 id	A-Z name	A-Z surname
1	1	Jack	Bauer

Mapeamento de Atributos

Agregação de dados 1-N

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.

```
Author jack = new Author();
jack.setName("Jack");
jack.setSurname("Bauer");
Address mainHouse =
    new Address ( street: "Rua Exemplo", city: "Lisboa", zipCode: "12345678");
Address vacHouse =
    new Address( street: "Travessa Feliz", city: "Cascais", zipCode: "421312312");
jack.addAddress(mainHouse);
jack.addAddress(vacHouse);
authorRepository.save(jack);
```

Esta linha regista o objecto *jack* para mapeamento.

1. Cria a linha na tabela author

author				
Grid	123 id	A-Z name	A-Z surname	
1	1	Jack	Bauer	

2. Porque Author é dona de Address, também são adicionados na tabela respectiva.

author_addresses				
Grid	123 author_id	A-Z city	A-Z street	A-Z zip_code
1	1	Lisboa	Rua Exemplo	12345678
2	1	Cascais	Travessa Feliz	421312312

Mapeamento de Atributos

Relações 1-1

@Entity

```
public class Author {
```

@OneToOne

```
private Debut debut;
```

Author é dono da
relação!



Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.

@Entity

```
public class Debut {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```
private Long id;
```

@OneToOne

```
private Book book;
```

```
private LocalDate date;
```

```
private String publisher;
```

Mapeamento de Atributos

Relações 1-1

@Entity

```
public class Author {
```

@OneToOne

```
private Debut debut;
```

@Entity

```
public class Debut {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```
private Long id;
```

@OneToOne

```
private Book book;
```

```
private LocalDate date;
```

```
private String publisher;
```

Padrão Foreign Key Mapping:

- Uma referência para outra entidade é mapeada para uma Foreign Key

author				
Enter a SQL expression to filter results (use Ctrl+Space)				
Grid	id	name	surname	debut_id
1	1	Jack	Bauer	1

debut				
Enter a SQL expression to filter results (use Ctrl+Space)				
Grid	id	date	publisher	book_id
1	1	2025-03-17	Black Horse	1

Mapeamento de Atributos

Relações 1-1

@Entity

```
public class Author {
```

@OneToOne

@JoinTable(name = "author_debut")

```
private Debut debut;
```

@Entity

```
public class Debut {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```
private Long id;
```

@OneToOne

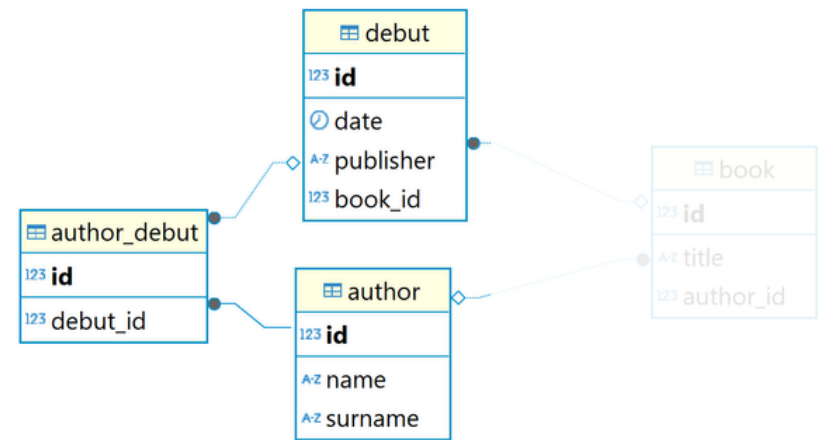
```
private Book book;
```

```
private LocalDate date;
```

```
private String publisher;
```

Alternativa **JoinTable**:

- É usada uma tabela auxiliar para fazer a associação 1-1.



Mapeamento de Atributos

Relações 1-1

@Entity

```
public class Author {
```

@OneToOne

@JoinTable(name = "author_debut")

```
private Debut debut;
```

@Entity

```
public class Debut {
```

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

```
private Long id;
```

@OneToOne

```
private Book book;
```

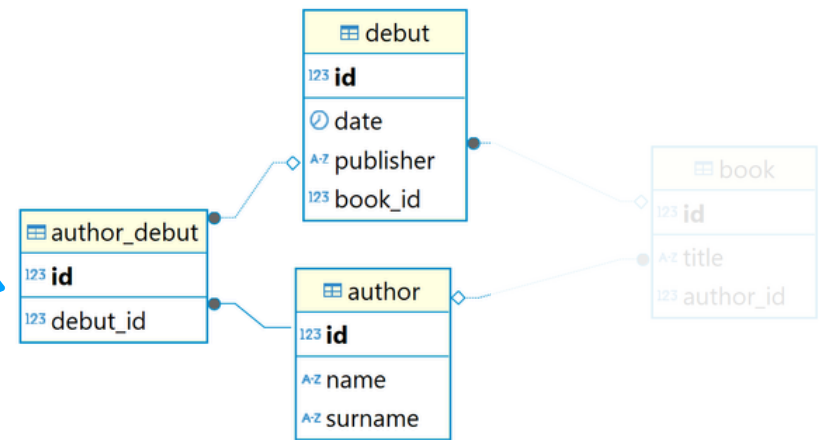
```
private LocalDate date;
```

```
private String publisher;
```

Cria uma tabela auxiliar para registrar a associação 1-1

Alternativa **JoinTable**:

- É usada uma tabela auxiliar para fazer a associação 1-1.



Análise:

- Ocupa mais espaço
- Menos eficiente (mais JOINS nos queries)
- + Maior independência das tabelas (para desenvolvimento corrente)

Mapeamento de Atributos

Relações 1-1 bidireccionais

```
@Entity
public class Author {

    @OneToOne(mappedBy = "author", cascade = CascadeType.ALL)
    @PrimaryKeyJoinColumn
    private Debut debut;
```

```
@Entity
public class Debut {
```

```
    @Id
    private Long id;
```

```
    @MapsId
    @OneToOne
    private Author author;
```

Padrão **Shared Primary Key**:

- As primary keys são partilhadas por ambas as tabelas.

author				
Enter a SQL expression to filter result				
Grid		id	name	surname
1		1	Jack	Bauer
2		2	Chloe	O'Brian
3		3	Kim	Bauer
4		4	David	Palmer
5		5	Michelle	Dessler
6		6	Machado	de Assis

debut					
Enter a SQL expression to filter results (use Ctrl+Space)					
Grid		date	publisher	123 author_id	123 book_id
1		2025-03-17	Black Horse	1	1
2		2025-03-17	Black Horse	6	2

Mapeamento de Atributos

Relações 1-1 bidireccionais

```
@Entity
public class Author {

    @OneToOne(mappedBy = "author", cascade = CascadeType.ALL)
    @PrimaryKeyJoinColumn
    private Debut debut;
```

```
@Entity
public class Debut {

    @Id
    private Long id;

    @MapsId
    @OneToOne
    private Author author;
```

Usa a mesma PK em ambas as tabelas

Padrão **Shared Primary Key**:

- As primary keys são partilhadas por ambas as tabelas.

author		Enter a SQL expression to filter result	
Grid	id	name	surname
1	1	Jack	Bauer
2	2	Chloe	O'Brian
3	3	Kim	Bauer
4	4	David	Palmer
5	5	Michelle	Dessler
6	6	Machado	de Assis

debut		Enter a SQL expression to filter result (use Ctrl+Space)		
Grid	date	publisher	author_id	book_id
1	2025-03-17	Black Horse	1	1
2	2025-03-17	Black Horse	6	2

Análise:

- SGDBs não têm como garantir a partilha de PKs
- + Ocupa menos espaço (menos uma coluna de FK)

Mapeamento de Atributos

Relações 1-1 bidireccionais

@Entity

```
public class Author {
```

@OneToOne

```
private Debut debut;
```

@Entity

```
public class Debut {
```

@Id

@GeneratedValue

```
private Long id;
```

@OneToOne(mappedBy = "debut")

```
private Author author;
```

Uma relação na BD para os atributos das duas classes

author					
Enter a SQL expression to filter results (use Ctrl+Space)					
Grid		123 id	A-Z name	A-Z surname	123 debut_id
1		1	Jack	Bauer	1

debut					
Enter a SQL expression to filter results (use Ctrl+Space)					
Grid		123 id	🕒 date	A-Z publisher	123 book_id
1		1	2025-03-17	Black Horse	1

Mapeamento de Atributos

Relações 1-1 bidireccionais

```
@Entity  
public class Author {
```

```
    @OneToOne  
    private Debut debut;
```

```
@Entity  
public class Debut {
```

```
    @Id  
    @GeneratedValue  
    private Long id;
```

```
    @OneToOne  
    private Author author;
```

← Classe dona da relação

```
Author jack = createJackBauer();  
Author machado = createMachado();
```

```
Book book = createBook(jack);  
jack.addBook(book);
```

```
Debut debut = new Debut(book,  
    LocalDate.now(),  
    publisher: "Black Horse");
```

```
debut.setAuthor(jack);  
machado.setDebut(debut);
```

Mapeamento de Atributos

Relações 1-1 bidireccionais

@Entity

```
public class Author {
```

@OneToOne

```
private Debut debut;
```

Classe dona da relação

@Entity

```
public class Debut {
```

@Id

@GeneratedValue

```
private Long id;
```

@OneToOne

```
private Author author;
```

Sem o mappedBy,
cria uma coluna
extra!

```
Author jack = createJackBauer();  
Author machado = createMachado();
```

```
Book book = createBook(jack);  
jack.addBook(book);
```

```
Debut debut = new Debut(book,  
    LocalDate.now(),  
    publisher: "Black Horse");
```

```
debut.setAuthor(jack);  
machado.setDebut(debut);
```

author Enter a SQL expression to filter results (use Ctrl+Space)

Grid	id	name	surname	debut_id
1	1	Jack	Bauer	[NULL]
2	2	Machado	de Assis	1

debut Enter a SQL expression to filter results (use Ctrl+Space)

Grid	id	date	publisher	author_id	book_id
1	1	2025-03-17	Black Horse	1	1

São duas relações completamente diferentes sem o mappedBy!
Pode levar a resultados inconsistentes quando usado erradamente.

Mapeamento de Atributos

Relações 1-N

@Entity

```
public class Author {
```

@OneToMany

```
private List<Book> books;
```

É criada uma tabela auxiliar

@Entity

```
public class Book {
```

@Id

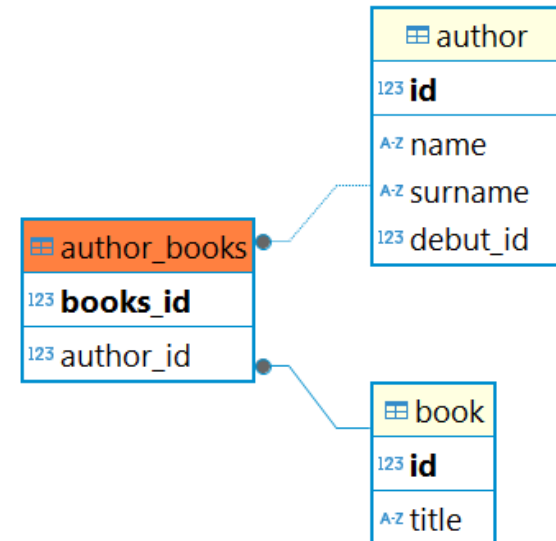
@GeneratedValue

```
private Long id;
```

```
private String title;
```

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.



Não é a forma mais eficiente de mapear 1-N!

Mapeamento de Atributos

Relações 1-N

@Entity

```
public class Author {
```

@OneToMany

@JoinColumn(name = "author_id")

```
private List<Book> books;
```

Cria uma coluna na
tabela Book

@Entity

```
public class Book {
```

@Id

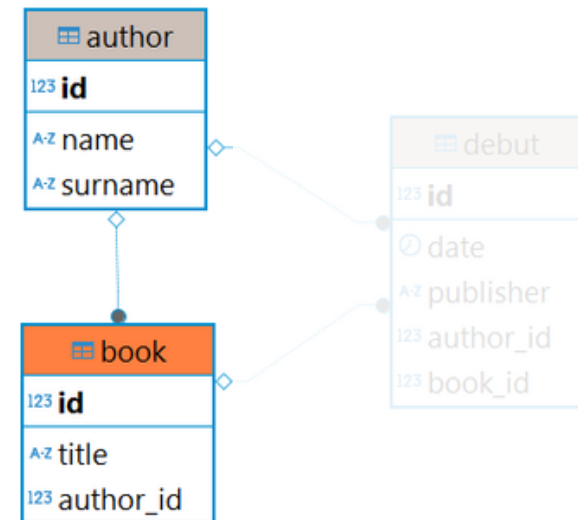
@GeneratedValue

```
private Long id;
```

```
private String title;
```

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.



É a forma mais eficiente de mapear 1-N!

Mapeamento de Atributos

Relações 1-N bidireccionais

@Entity

```
public class Author {
```

```
    @OneToMany(mappedBy = "escritor")
```

```
    private List<Book> books;
```

Indica que esta relação é a mesma que a de escritor para Author.

@Entity

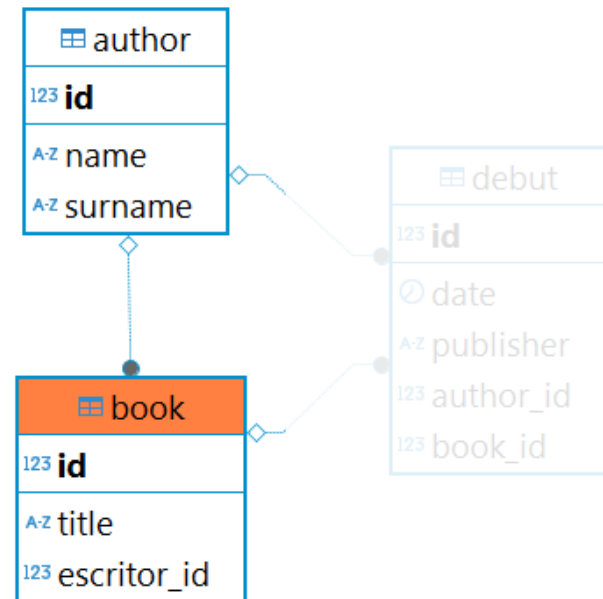
```
public class Book {
```

```
    @ManyToOne
```

```
    private Author escritor;
```

Padrão **Dependent Mapping**:

- A classe dona trata do mapeamento dos seus dados/objetos que agrega.



Mapeamento de Atributos

Relações M-N

```
@Entity
public class Book {

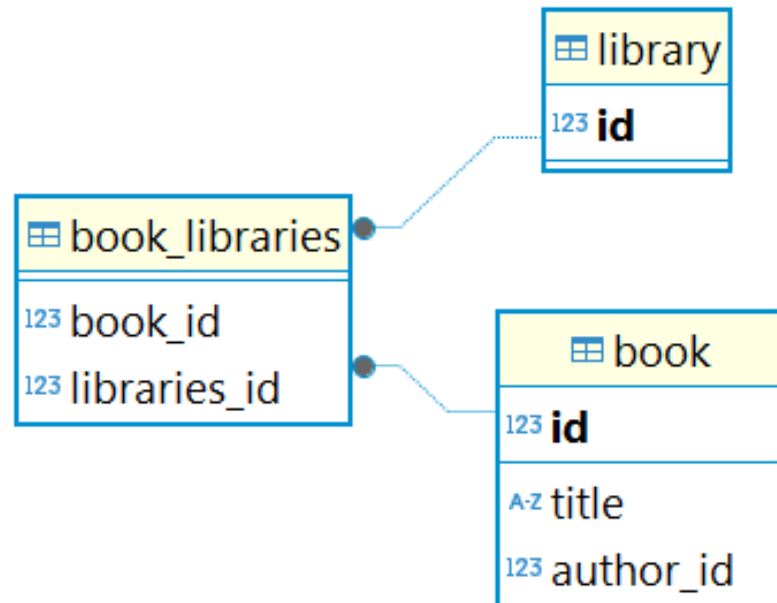
    @ManyToMany
    private List<Library> libraries;
```

```
@Entity
public class Library {

    @Id
    @GeneratedValue
    long id;
```

Padrão **Association Table Mapping**:

- Cria-se uma tabela para fazer mapeamento de M-N.



Mapeamento de Atributos

Relações M-N bidireccional

```
@Entity
public class Book {

    @ManyToMany
    private List<Library> libraries;
```

```
@Entity
public class Library {

    @ManyToMany
    private List<Book> books;
```

Padrão **Association Table Mapping**:

- Cria-se uma tabela para fazer mapeamento de M-N.

Mapeamento de Atributos

Relações M-N bidireccional

```
@Entity
public class Book {

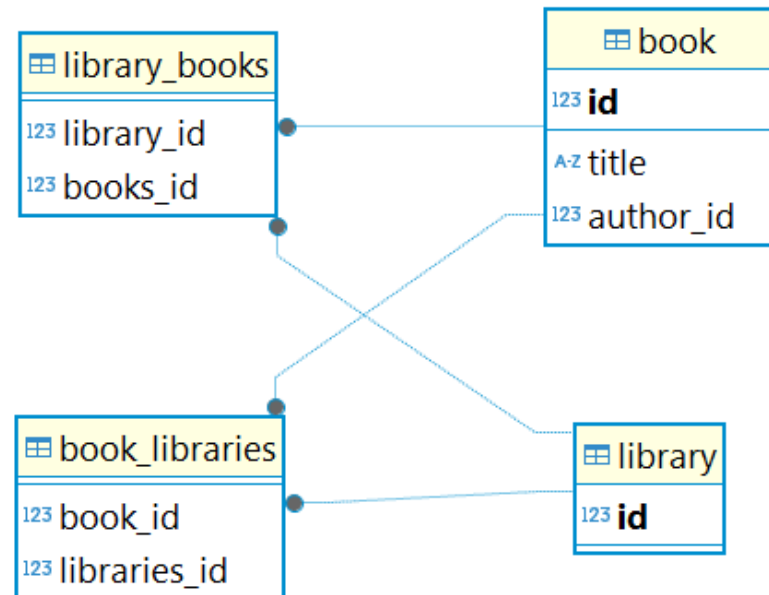
    @ManyToMany
    private List<Library> libraries;
```

```
@Entity
public class Library {
```

```
    @ManyToMany
    private List<Book> books;
```

Padrão **Association Table Mapping**:

- Cria-se uma tabela para fazer mapeamento de M-N.



Mapeamento de Atributos

Relações M-N bidireccional

@Entity

```
public class Book {
```

@ManyToMany

```
private List<Library> libraries;
```

Indica que esta relação é a mesma que a de libraries para Library.

@Entity

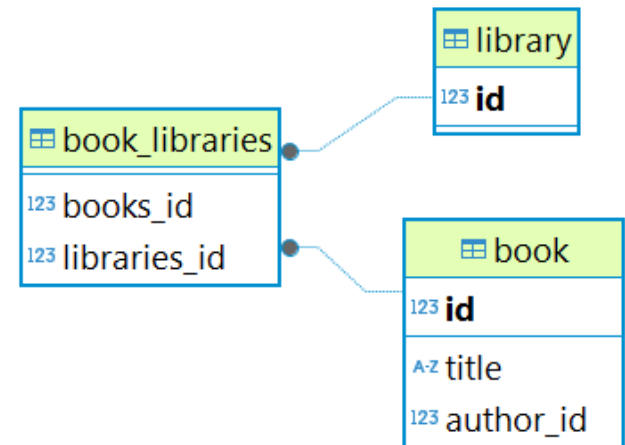
```
public class Library {
```

```
@ManyToMany(mappedBy = "libraries")
```

```
private List<Book> books;
```

Padrão **Association Table Mapping**:

- Cria-se uma tabela para fazer mapeamento de M-N.



Uni vs Bidirecional

```
@ManyToMany
@JoinTable(
    name = "course_like",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
Set<Course> likedCourses;
```



```
@ManyToMany(mappedBy = "likedCourses")
Set<Student> likes;
```

```
@Entity
@Table(name="CART")
public class Cart {

    //...

    @OneToMany(mappedBy="cart")
    private Set<Item> items;

    // getters and setters
}
```

- A conhece uma lista de Bs. B conhece uma lista de As. Será a mesma relação?
- Uma pessoa pode estar inscrita numa lista de disciplinas. Uma disciplina tem como corpo docente uma lista de pessoas. São diferentes!
- Uma pessoa pode estar inscrita numa lista de disciplinas. Cada disciplina tem uma lista de alunos inscrito. São iguais!

Associações entre objectos

- `@OneToOne`, `@OneToMany`, `@ManyToMany` definem como a relação é mapeada em tabelas
- `Collection<T>` precisa de ser anotado para saber se é de 1-N ou N-N
- `@JoinColumn` indica que a relação 1-N vai existir como FK na outra classe (`T`)
- Por omissão (na ausência de `@JoinColumn`), é sempre criada uma tabela extra. Logo `@JoinColumn` é praticamente obrigatória

Classe dona da relação

- A classe dona é responsável por actualizar a relação (evitando duplicação)
- `@ManyToOne` é a dona da relação (=tem a FK)
- `@OneToMany` não é dona da relação (=não tem a FK)
- O `mappedby` fica do lado “fraco” da relação (=não tem a FK)
- A anotação `@JoinColumn` indica o nome da coluna com a FK
 - Fica do lado fraco da relação

Outras anotações JPA

- Definição de propriedades de colunas extra:
`@Column(name="curso", nullable=false, length=512)`
- Atributos que não vão para tabelas (ex: cache):
`@Transient`
- Persistir mapas directamente: `@MapKeyEnumerated` e `@MapKeyTemporal` (Enums e Datetimes)
- Persistir dados binários (ex: imagens, ficheiros): `@Lob`

Mais informações

- <https://howtodoinjava.com/hibernate/hibernate-one-to-one-mapping/>
- <https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa>

Object-Relational Mapping

Parte 2 - Mapeamento de Relações